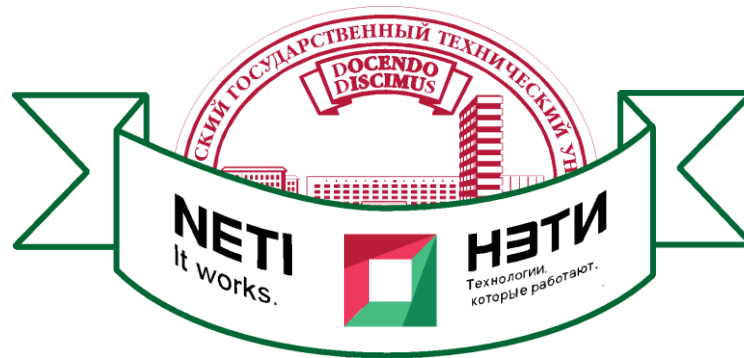


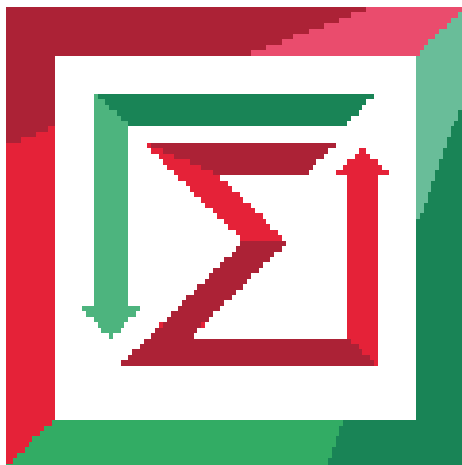
Министерство науки и высшего образования
Российской Федерации

Федеральное государственное бюджетное
образовательное учреждение высшего образования
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»



Кафедра теоретической и прикладной информатики

Современные Компьютерные Технологии
«Обратная задача гравirazведки»



Факультет:	ФПМИ
Группа:	ПММ-51
Вариант:	5
Студенты:	Ковалевский Дмитрий Панасенко Сергей Кожин Матвей
Преподаватели:	Домников П.А.

Новосибирск
2026

0. Вариант

Разработать программу решения обратной задачи гравиразведки. В качестве измеряемых данных использовать компоненту g_z вектора силы тяжести.

Регуляризация по γ . В качестве решения прямой задачи использовать аналитическое решение.

1. Теоретическая часть

Гравитационное поле описывается уравнением

$$\operatorname{div} \vec{G} = \rho$$

где ρ - плотность, а $\vec{G}$ - вектор силы тяжести

В произвольной точке М поле точечного источника с мощностью ρ , расположенного в начале координат, определяется соотношением

$$\vec{G}^k = \frac{\rho}{4\pi r^3} (\tilde{x}\vec{e}_x + \tilde{y}\vec{e}_y + \tilde{z}\vec{e}_z),$$

где r - это расстояние от точечного источника до точки М с координатами (x, y, z) , $\vec{e}_x, \vec{e}_y, \vec{e}_z$ - соответствующие орты системы координат, а ρ - мощность источника.

Таким образом, если всю расчетную область Ω представить в виде объединения непересекающихся областей $\tilde{\Omega}_k$, то искомое поле может быть найдено $\vec{G} = \sum \rho_k \vec{G}^k$, где вектор $\vec{G}^k = (\vec{G}_x^k, \vec{G}_y^k, \vec{G}_z^k)$ определяется соотношением

$$\vec{G}^k = \operatorname{mes}(\tilde{\Omega}_k) \frac{1}{4\pi r^3} (\tilde{x}\vec{e}_x + \tilde{y}\vec{e}_y + \tilde{z}\vec{e}_z).$$

Величина ρ_k - это среднее значение ρ внутри $\tilde{\Omega}_k$, (x, y, z) , - значения координат точки М при переносе начала системы координат в центр $\tilde{\Omega}_k$, а $\operatorname{mes}(\tilde{\Omega}_k)$, соответственно, объем области $\tilde{\Omega}_k$.

На практике, как правило, измеряется только вертикальная компонента вектора $\vec{G}$, поэтому в дальнейших рассуждениях будем рассматривать только приращение компоненты g_z , которое будем обозначать Δg .

Значения избыточной ρ будут определяться на основании минимизации функционала, представляющего собой сумму взвешенных квадратов невязки практических и теоретических данных:

$$\Phi(\rho) = \sum_{i=1}^n (\Delta g_i - \Delta \tilde{g}_i)^2$$

где Δg_i - значения приращений вертикальной компоненты $\vec{G}$, полученные на -м приемнике ($i = 1 \dots n$, n – количество приемников), а ρ - искомый вектор параметров избыточной плотности, компонентами которого являются значения плотности в ячейках Ω_k исследуемой области.

Значения $\Delta \tilde{g}_i$, как уже говорилось выше, могут быть вычислены в виде

$$\Delta \tilde{g}_i = \sum_{k=1}^K \rho_k \Delta \tilde{g}_i^k$$

где K - количество ячеек.

Минимизация функционала эквивалентна решению СЛАУ (размером $K \times K$)

$$A\rho = b$$

где компоненты матрицы A и вектора b определяются соотношениями

$$A_{qs} = \sum_{i=1}^n \Delta \tilde{g}_i^q * \Delta \tilde{g}_i^s$$

$$b_q = \Delta \tilde{g}_i^q * \Delta g_i$$

Однако при большом числе ячеек задача становится плохо обусловленной и чувствительной к шуму в данных. Для повышения устойчивости решения применяется регуляризация. В данной работе используется регуляризация по γ :

$$\Phi^P(\rho) = \sum_{i=1}^n (\Delta g_i - \Delta \tilde{g}_i)^2 + \sum_{k=1}^K [\gamma_k \sum (\rho_k - \rho_m)^2]$$

Минимизация данного функционала эквивалентна решению СЛАУ

$$(A + C)\rho = b$$

где C – матрица, определяющая соотношениями

$$C_{ij} = -(\gamma_i + \gamma_j)$$

для которых j - номер ячейки, соседней с i -й ячейкой,

$$C_{ii} = \gamma_i \sum_m 1 + \sum_m \gamma_m$$

m - номера всех соседних с i -й ячеек.

2. Исследование

Расчетная область дискретизирована в двумерную сетку из прямоугольных ячеек размером 1×1 условных единиц. Область покрывает 40 единиц по горизонтали, вдоль оси x , и 20 единиц по вертикали, вдоль оси z . Таким образом, расчетная зона состоит из 40 ячеек по x и 20 ячеек по z , всего 800 ячеек.

Для задания плотностной аномалии используется прямоугольная область внутри расчетной сетки. Ячейки с индексами по x от 8 до 11 и по z от 3 до 5 получают значение плотности $\rho=1.0$. Остальные ячейки имеют нулевую плотность. Таким образом, аномалия занимает 12 ячеек и расположена внутри расчетной области на некоторой глубине.

Система наблюдений представляет собой линейный профиль из 100 приемников, равномерно распределенных вдоль оси x от 0 до 40 условных единиц. Все приемники расположены на уровне $z=1.0$. В качестве измеряемых данных используется вертикальная компонента поля Δg .

Для проверки устойчивости решения к практическим данным добавляется малый случайный шум с амплитудой 10^{-2} . Обратная задача решается с использованием регуляризации по параметру γ . После восстановления распределения плотности вычисляются расчетные значения поля, функционал невязки, а также строятся графики наблюдаемых и расчетных данных, график невязки и распределения истинной и восстановленной плотности.

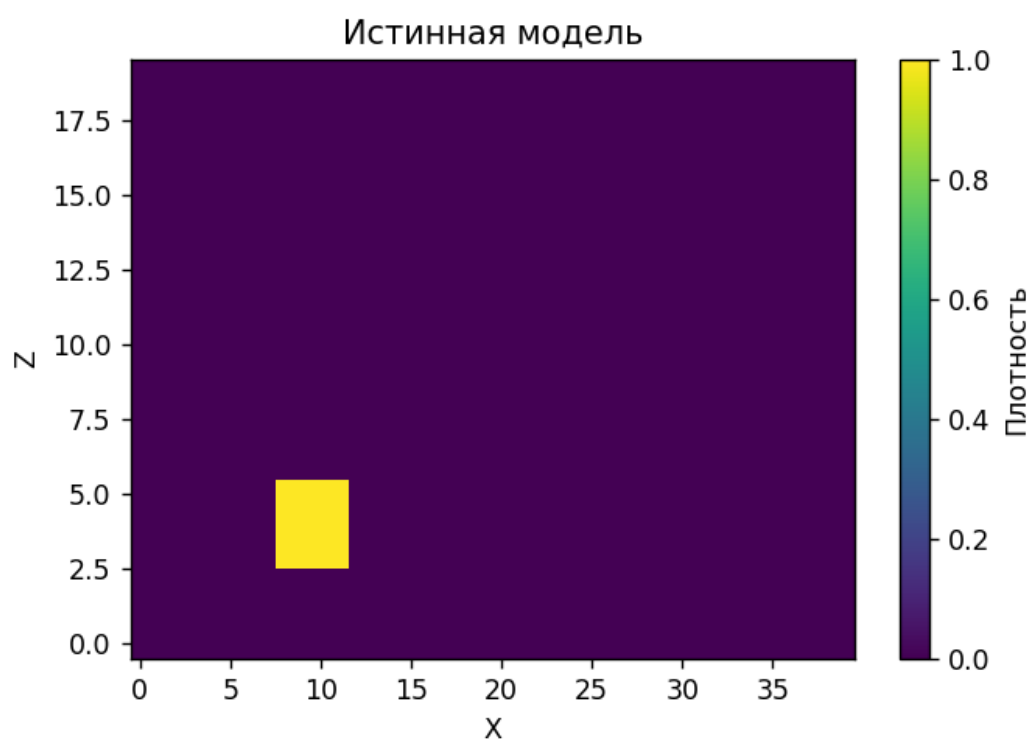


Рисунок 1 – Истинная модель

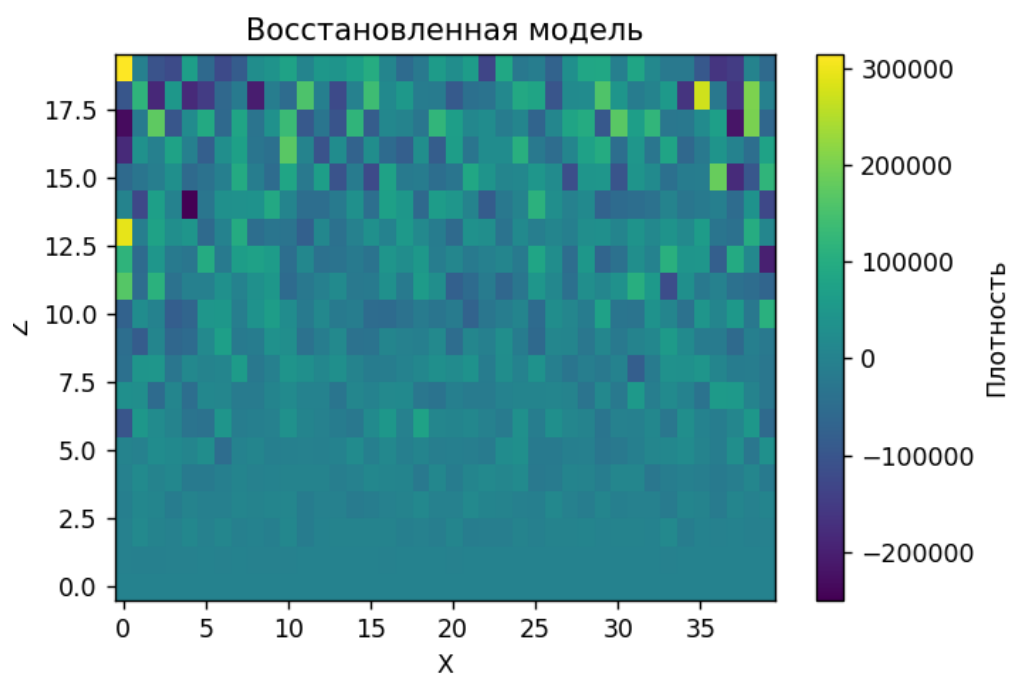


Рисунок 2 – Восстановленная модель без регуляризации ($\gamma = 0$)



Рисунок 3 – Восстановленная модель со слабой регуляризацией ($\gamma = 1e - 5$)



Рисунок 4 – Восстановленная модель с сильной регуляризацией ($\gamma = 0.1$)

3. Вывод

В ходе работы была реализована программа решения обратной задачи гравиразведки с использованием линейной инверсии и регуляризации по параметру γ . Расчетная область была дискретизирована на ячеистую структуру, для которой задавалось распределение плотности и моделировались синтетические данные. По полученным данным была успешно восстановлена модель аномалии.

Проведенные вычислительные эксперименты показали, что без регуляризации решение является неустойчивым и содержит значительные осцилляции, тогда как введение γ -регуляризации позволяет получить сглаженное и физически осмысленное распределение плотности. Сравнение практических и расчетных данных продемонстрировало малую невязку, что подтверждает корректность реализованного алгоритма.

4. Текст программы

```
import numpy as np
import matplotlib.pyplot as plt

# 1. ПАРАМЕТРЫ СЕТКИ
nx, nz = 40, 20 # количество ячеек
dx, dz = 1.0, 1.0

cells = []
for ix in range(nx):
    for iz in range(nz):
        x = ix * dx
        z = -iz * dz # вниз
        cells.append((x, z))

n_cells = len(cells)

# 2. ПРИЕМНИКИ (профиль)
n_receivers = 100
receivers = [(x, 1.0) for x in np.linspace(0, nx*dx, n_receivers)]

# 3. ПРЯМАЯ ЗАДАЧА
def compute_g(cell, receiver):
    x, z = cell
    xr, zr = receiver
```

```

dx = xr - x
dz = zr - z

r = np.sqrt(dx**2 + dz**2) + 1e-8

return dz / (r**3) # вертикальная компонента

```

4. МАТРИЦА L

```

def build_L(cells, receivers):
    L = np.zeros((len(receivers), len(cells)))

    for i, r in enumerate(receivers):
        for j, c in enumerate(cells):
            L[i, j] = compute_g(c, r)

    return L

```

5. СОСЕДИ (для γ)

```

def get_neighbors(nx, nz):
    neighbors = [[] for _ in range(nx * nz)]

    def index(ix, iz):
        return iz + ix * nz

    for ix in range(nx):
        for iz in range(nz):
            i = index(ix, iz)

            if ix > 0:
                neighbors[i].append(index(ix - 1, iz))
            if ix < nx - 1:
                neighbors[i].append(index(ix + 1, iz))
            if iz > 0:
                neighbors[i].append(index(ix, iz - 1))
            if iz < nz - 1:
                neighbors[i].append(index(ix, iz + 1))

    return neighbors

```

6. МАТРИЦА РЕГУЛЯРИЗАЦИИ

```

def build_C(n_cells, neighbors, gamma):

```



```

C = np.zeros((n_cells, n_cells))

for i in range(n_cells):
    for j in neighbors[i]:
        C[i, i] += gamma
        C[i, j] -= gamma

return C

# 7. ОБРАТНАЯ ЗАДАЧА
def solve_inverse(L, g, C):
    A = L.T @ L
    b = L.T @ g

    return np.linalg.solve(A + C, b)

# 8. СОЗДАНИЕ ИСТИННОЙ МОДЕЛИ
true_rho = np.zeros(n_cells)

# задаем аномалию (прямоугольник)
for ix in range(8, 12):
    for iz in range(3, 6):
        idx = iz + ix * nz
        true_rho[idx] = 1.0

# 9. РАСЧЕТ ДАННЫХ
L = build_L(cells, receivers)
g_true = L @ true_rho

# добавим небольшой шум
noise = 0.01 * np.random.randn(len(g_true))
g_obs = g_true + noise
# g_obs = g_true # без шума для наглядности

# 10. РЕГУЛЯРИЗАЦИЯ  $\gamma$ 
neighbors = get_neighbors(nx, nz)
gamma = 1e-1

C = build_C(n_cells, neighbors, gamma)

```

```
# 11. РЕШЕНИЕ
```

```
rho_est = solve_inverse(L, g_obs, C)
```

```
# 12. РАСЧЕТ МОДЕЛЬНОГО СИГНАЛА
```

```
g_calc = L @ rho_est
```

```
# 13. НЕВЯЗКА
```

```
misfit = np.linalg.norm(g_obs - g_calc)
```

```
print("Невязка:", misfit)
```

```
# 14. ВИЗУАЛИЗАЦИЯ
```

```
# --- поле ---
```

```
plt.figure(figsize=(10, 4))
```

```
plt.plot(g_obs, label="Наблюдаемые")
```

```
plt.plot(g_calc, '--', label="Рассчитанные")
```

```
plt.legend()
```

```
plt.title("Сигналы вдоль профиля")
```

```
plt.xlabel("Приемник")
```

```
plt.ylabel("Δg")
```

```
plt.grid()
```

```
# --- невязка ---
```

```
plt.figure(figsize=(10, 4))
```

```
plt.plot(g_obs - g_calc)
```

```
plt.title("Невязка")
```

```
plt.grid()
```

```
# --- распределение плотности ---
```

```
def plot_model(rho, title):
```

```
    model = rho.reshape((nx, nz)).T
```

```
    plt.figure(figsize=(6, 4))
```

```
    plt.imshow(model, origin='lower', aspect='auto')
```

```
    plt.colorbar(label="Плотность")
```

```
    plt.title(title)
```

```
    plt.xlabel("X")
```

```
    plt.ylabel("Z")
```

```
plot_model(true_rho, "Истинная модель")
```

```
plot_model(rho_est, "Восстановленная модель")
```

```
plt.show()
```